ADZE: Allelic Diversity Analyzer
Version 2.0

Zachary A. Szpiech Mattias Jakobsson Noah A. Rosenberg

Abstract

ADZE implements a rarefaction approach to estimating allelic richness, private allelic richness, and private allelic richness for combinations of populations, correcting for differences in sample size between populations. Version 2.0 computes the same estimators as version 1.0, and at every sample size that version 1.0 reports it gives the same values. In addition, version 2.0 reads the variant files produced by current sequencing studies, can report each statistic in windows along the genome and for haplotypes of consecutive SNPs, and is run from the command line rather than from a parameter file. It also uses much less memory and time than version 1.0. Section 10 lists everything that changed.

Contents

1 Introduction **3**
- 1.1 Allelic richness . . . 3
- 1.2 Private allelic richness . . . 4
- 1.3 Private allelic richness for a combination of populations . . . 4
- 1.4 Summary statistics . . . 5
- 1.5 One gene copy . . . 5

2 Getting started **6**
- 2.1 Obtaining ADZE . . . 6
- 2.2 Building . . . 6
- 2.3 Running . . . 6

3 Input **6**
- 3.1 Choosing a format . . . 6
- 3.2 STRUCTURE format . . . 7
- 3.3 Dimensions of the data file . . . 7
- 3.4 VCF format . . . 8
- 3.5 Groupings . . . 9
- 3.6 Missing data and locus filtering . . . 9
- 3.7 How a run proceeds . . . 9

4 Parameters and options **10**
- 4.1 Parameter file syntax . . . 10
- 4.2 Reporting a single g . . . 11
- 4.3 Option reference . . . 11

4.3.1 Input options . . . 11
4.3.2 Analysis options . . . 13
4.3.3 Output options . . . 14
4.3.4 Other options . . . 14

5 Output **15**
5.1 Allelic richness and private allelic richness . . . 15
5.2 Private allelic richness of tuples . . . 15
5.3 Locus-specific output . . . 16
5.4 Filtered loci . . . 16
5.5 Run summary . . . 17
5.6 Version 1.0's layout . . . 17
5.7 Exit status . . . 17

6 Sliding windows **17**
6.1 What a windowed statistic is here . . . 17
6.2 Coordinates . . . 18
6.3 Defining the windows . . . 18
6.4 g is fixed across the scan . . . 18
6.5 Output . . . 19

7 Haplotype alleles **19**
7.1 What a haplotype allele is here . . . 19
7.2 Phase is required . . . 20
7.3 One uncalled genotype makes the block uncalled . . . 20
7.4 Richness saturates as blocks lengthen . . . 20
7.5 Naming and output . . . 21
7.6 Haplotype blocks in sliding windows . . . 21

8 Examples **22**
8.1 The distributed example data set . . . 22
8.2 A larger data set . . . 22
8.3 Naming the tuples of interest . . . 26
8.4 A genome scan . . . 26

9 Performance and threading **27**
9.1 Haplotype blocks . . . 28

10 Changes from version 1.0 **29**
10.1 Compatibility . . . 29
10.2 Interface . . . 29
10.3 Behaviour . . . 30
10.4 Verification . . . 31

References **31**

1 Introduction

The analysis of the distributions of alleles across populations is important for elucidating genetic diversity and population relationships. Two fundamental quantities for a population at a given locus are the number of distinct alleles in the population and the number of alleles private to the population (that is, not found in other populations). These quantities are especially informative when populations are studied for highly variable multiallelic markers, such as microsatellites.

The number of distinct alleles and the number of private alleles depend heavily on sample size, and they can be difficult to interpret when sample sizes differ across populations. The rarefaction approach has been an important strategy for producing estimates that are comparable in different populations (Hurlbert, 1971; Petit et al., 1998; Kalinowski, 2004, 2005). The idea of the rarefaction method is to trim unequal samples to the same standardized sample size, a number less than or equal to the smallest sample size across populations. For a standardized size g, populations are compared by considering the estimates of "allelic richness" and "private allelic richness" that would be obtained when averaging across all subsamples of size g. In the rarefaction framework, the estimated allelic richness of a population is the number of distinct alleles expected in a random subsample of size g drawn from the population (Hurlbert, 1971; Petit et al., 1998). The estimated private allelic richness is the number of private alleles expected in the population when random subsamples of size g are taken from each of J populations under consideration (Kalinowski, 2004). Combinatorial formulas make it possible to compute these statistics relatively quickly. Note that these are exact expectations under sampling without replacement rather than averages over random subsamples, so the results do not depend on a random seed.

Often, as was noted by Kalinowski (2004), especially if several populations in a sample are closely related, few alleles are private to individual populations. Instead, alleles may be private to groups of populations (that is, alleles may be found in each of several members of a larger set of populations). We therefore introduce a generalization of the private allelic richness concept of Kalinowski (2004). We compute a generalized private allelic richness statistic that uses a rarefaction approach to measure the number of distinct alleles private to a group of populations and found in all populations in the group. This statistic makes it possible to evaluate the sample size-corrected number of distinct alleles private to any set of populations, and it reduces to private allelic richness when the group of populations consists of only a single population. We demonstrate the application of the new generalized private allelic richness statistic using microsatellite genotypes from human populations (Szpiech et al., 2008). By considering the sample size-corrected number of distinct alleles private to various combinations of major geographic regions, this analysis produces evidence in support of the hypothesis that an early human migration from Africa to Oceania did not have an appreciable effect on genetic variation in modern populations of Asia. We have developed a computer program, ADZE (a tool for "chopping" samples down to standardized sizes for data analysis), that implements computations of allelic richness, private allelic richness, and our new measure of generalized private allelic richness.

1.1 Allelic richness

Consider a locus with I distinct alleles, and define N_{ij} as the number of copies of allele type i in a sample from population j. $N_j = \sum_{i=1}^{I} N_{ij}$ is the sample size of population j at the locus. The probability of finding no copies of allele type i in a subsample of g gene copies from population j is

$$Q_{ijg} = \frac{\binom{N_j - N_{ij}}{g}}{\binom{N_j}{g}}. \tag{1}$$

Then the probability of finding at least one copy of allele type i in a sample of size g alleles from population j is $P_{ijg} = 1 - Q_{ijg}$, and

$$\hat{\alpha}_g^{(j)} = \sum_{i=1}^{I} P_{ijg} \tag{2}$$

is the estimated allelic richness of a sample of size g from population j (Hurlbert, 1971; Petit et al., 1998; Kalinowski, 2004). Equation 2 estimates the expected number of distinct alleles that will be observed in population j in a sample of size g.

1.2 Private allelic richness

Using this notation, the estimated private allelic richness for a sample size g from population j can be written as

$$\hat{\pi}_g^{(j)} = \sum_{i=1}^{I} \left[P_{ijg} \left(\prod_{\substack{j'=1 \\ j' \neq j}}^{J} Q_{ij'g} \right) \right], \tag{3}$$

where J is the total number of populations (Kalinowski, 2004). This formula sums over distinct allele types, i, the probability that a random subsample of size g from population j contains allele type i and that subsamples of size g from the remaining populations do not contain i.

1.3 Private allelic richness for a combination of populations

Generalizing the concept of private allelic richness, we can consider the number of distinct alleles private to some combination of k populations selected from $\{1, 2, \ldots, J\}$. Consider a set of J populations labeled 1 to J, and let $S = \{1, 2, \ldots, J\}$. Let C_k be the set of all possible combinations of k elements from S. There are $\binom{J}{k}$ possible combinations in C_k. We label these combinations by C_{km}, where m ranges from 1 to $\binom{J}{k}$. Using the following equation we can calculate $\hat{\pi}_{gk}^{(m)}$, the estimated number of distinct alleles private to the mth combination of k populations, when samples of size g are drawn from each of the J populations:

$$\hat{\pi}_{gk}^{(m)} = \sum_{i=1}^{I} \left[\left(\prod_{j \in C_{km}} P_{ijg} \right) \left(\prod_{j' \in S \setminus C_{km}} Q_{ij'g} \right) \right]. \tag{4}$$

$S \setminus C_{km}$ denotes the set S excluding the elements of C_{km}. For $k = 1$, $\hat{\pi}_{gk}^{(m)}$ reduces to private allelic richness as in Equation 3. For $k = J - 1$, Equation 4 can be considered a measure for "missing allelic richness", and it reduces to

$$\hat{\mu}_g^{(j)} = \sum_{i=1}^{I} \left[Q_{ijg} \left(\prod_{\substack{j'=1 \\ j' \neq j}}^{J} P_{ij'g} \right) \right]. \tag{5}$$

In this equation, $\hat{\mu}_g^{(j)}$ gives a sample size-corrected measure of the number of distinct alleles found in all populations other than population j.

1.4 Summary statistics

ADZE implements these calculations (Equations 2–4) at each of many loci in a data set, and its primary method of reporting results gives the mean, variance, and standard error of the mean across these loci. Section 5.3 has information on how to output locus-specific estimates in addition to the summary statistics. The mean across loci is calculated at each g and for each population (or population grouping) in the following manner:

$$\bar{x} = \frac{1}{L}\sum_{\ell=1}^{L} x_\ell, \tag{6}$$

where L is the total number of loci used in the calculations, and x_ℓ is one of the statistics described in the above sections calculated at locus ℓ. Variance across loci is calculated at each g and for each grouping as

$$s^2 = \frac{1}{L-1}\sum_{\ell=1}^{L}(x_\ell - \bar{x})^2, \tag{7}$$

and standard error is calculated at each g and for each grouping as

$$\mathrm{SE} = \frac{s}{\sqrt{L}}. \tag{8}$$

Note that a statistic is only defined at a given g if it is defined at that g for every locus used. For allelic richness this requires $g \leq N_j$ at each locus. For private allelic richness, g can be no larger than the smallest N_j across populations at each locus. Where a statistic is not defined, ADZE reports `NA` (section 5.6). With a single locus ($L = 1$), the variance and standard error are undefined and are reported as `nan`.

1.5 One gene copy

ADZE reports each statistic starting at $g = 1$. At this sample size the statistics have simple closed forms, and allelic richness takes the same value at every locus.

At $g = 1$, $Q_{ij1} = (N_j - N_{ij})/N_j$, so $P_{ij1} = N_{ij}/N_j = p_{ij}$, the frequency of allele type i in population j. Allelic richness is then $\sum_i p_{ij} = 1$ at every locus, because a single gene copy always carries exactly one allele. This value says nothing about the data, but it is a useful check, since a locus where it is not 1 indicates a problem with how the alleles were counted. The variance across loci is reported as a number near 10^{-33}. This is rounding error in a sum of ones, and it is zero for practical purposes.

Private allelic richness at $g = 1$ is $\hat{\pi}_1^{(j)} = \sum_i p_{ij} \prod_{j' \neq j}(1 - p_{ij'})$, the probability that a single gene copy drawn from population j carries an allele that is missed by a single draw from each of the other populations. Note that every allele found in population j contributes to this sum, weighted by the probability that it is missed in the other populations. It is therefore not the same as the summed frequency of the alleles private to population j. Private allelic richness for a combination of populations at $g = 1$ has the same form, with the product of p_{ij} taken over the populations in the combination.

2 Getting started

2.1 Obtaining adze

Source code is available at `https://github.com/szpiech/ADZE`. Version 2.0 requires only a C++17 compiler. If zlib is available, it is used to read compressed input. Without zlib, the program builds and runs, and a `.gz` file is refused with an error message instead of being misread. `--threads` uses the threads of the C++ standard library, so it needs no other library.

Use the following citation for ADZE: Szpiech, Z. A., M. Jakobsson, and N. A. Rosenberg. 2008. ADZE: a rarefaction approach for counting alleles private to combinations of populations. *Bioinformatics* 24: 2498–2504.

2.2 Building

With CMake:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
cmake --install build --prefix ~/.local      # optional
```

Or with plain `make`:

```
make -C src                  # no compressed input
make -C src ZLIB=1           # with compressed input
```

CMake reports whether it found zlib when it configures the build. The `make` build takes it as a flag.

2.3 Running

There are two ways to specify a run, and they can be mixed:

```
adze --data mydata.stru --group-col 5 --out-prefix mypops
adze myparamfile.txt --tolerance 0.05
```

The first form takes every parameter from the command line. The second reads a parameter file in the version 1.0 format, with the same keywords and the same short flags, and any option given on the command line takes precedence over the file. In other words, parameters are taken first from the built-in defaults, then from the parameter file if one is given, and then from the command line, with each overriding the one before.

`--help` lists every option, and `--version` prints the version and the citation for the method. Both work without a data file or a parameter file. `--write-template` writes a commented parameter file that can be run as it is once `DATA_FILE` is filled in.

Only the data file is required. Every other parameter either has a default or is measured from the data (section 3.3).

3 Input

3.1 Choosing a format

ADZE reads two input formats: the STRUCTURE format used by version 1.0 (section 3.2), and VCF (section 3.4), which can be compressed with `gzip` or `bgzip`. `--format` selects the format. Its

default, `auto`, decides from the file name, so that a file whose name ends in `.vcf`, `.vcf.gz` or `.vcf.bgz` is read as VCF.

Both formats are reduced to the same quantities at each locus, the allele counts and the missing-data counts for each grouping, and these are all that the estimators use. The results therefore do not depend on the input format. The same genotypes written in both formats give byte-identical output, and `test/formats.py` checks that they do.

3.2 structure format

The data file is in the same format used by STRUCTURE (Figure 1). The first line is a listing of locus names, optionally followed by a specified number of non-data lines. Then each individual is printed on two consecutive lines (for haploid or polyploid data, the number of lines per individual may differ). Immediately preceding the data on every line is a specified number of columns that contain classifications of the individual. Allele representation is not limited to numbers, and any string without whitespace can serve to represent an allele.

Figure 1 shows `example/small_data.stru`, which is distributed with ADZE. It contains five loci for eleven diploid individuals, and missing alleles are coded `-9`. The five columns before the genotype data represent (1) individual code numbers, (2) population code numbers, (3) population names, (4) countries of origin, and (5) geographic regions of origin. Grouping by the fifth column gives three groupings.

```
D12S1638 D14S1007 D9S1779 D9S1825 D7S2477
1037 87 Pima Mexico AMERICA 126 128 124 -9 142
1037 87 Pima Mexico AMERICA 126 128 142 -9 142
1038 87 Pima Mexico AMERICA 120 124 124 129 142
1038 87 Pima Mexico AMERICA 126 128 152 137 142
   ...
1357 24 Basque France EUROPE 126 120 124 -9 154
1357 24 Basque France EUROPE 128 122 126 -9 156
   ...
747 684 Japanese Japan EAST_ASIA 126 -9 124 129 158
747 684 Japanese Japan EAST_ASIA 128 -9 144 135 158
   ...
```

Figure 1: The distributed example data file, abridged. Row 1 holds the locus names, and each subsequent row is one gene copy, with five label columns followed by five allele codes.

3.3 Dimensions of the data file

Version 1.0 required `LOCI`, `DATA_LINES`, `NON_DATA_ROWS` and `NON_DATA_COLS` to be given in the parameter file, and it used them to check the data file. Version 2.0 measures them as it reads the file:

`--loci`
  is the number of names in the header row.

`--data-lines`
  is the number of data rows.

`--non-data-cols`
is the number of fields in the first data row minus the number of loci.

`--group-col`
is the last label column by default.

`--non-data-rows`
is 1 by default, for the row of locus names. This is the one value that cannot be measured, so it must be given if the file has additional header rows.

`--max-g`
is by default the largest standardized sample size the data supports. This is the smallest N_j over all groupings and all loci that remain after missing-data filtering. It is computed after filtering because dropping loci can raise it.

Any of these can still be given. In that case the given value is used, and a warning is printed if it differs from the measured value. `--dry-run` reports the measured dimensions, the sample size of each grouping, the largest feasible `MAX_G` and the number of tuples an analysis would evaluate, and then exits without computing any statistics.

3.4 VCF format

Each VCF record is one locus, and each allele index is one allele type, so the reference allele is allele 0 and the alternate alleles follow. A sample contributes as many gene copies as its `GT` field holds, so haploid and diploid records can be in the same file. A `.` is an uncalled gene copy. A sample's ploidy is the largest number of alleles held by any of its `GT` fields, since that is the number of gene copies it could contribute. At a record where the sample has fewer alleles (for example, a haploid call in an otherwise diploid file), the difference is counted as missing, and so it counts against that locus under `--tolerance`.

Only `GT` is read. No other `FORMAT` field, and no `INFO`, `QUAL` or `FILTER` value, affects the counts, so any filtering must be done before the file is given to ADZE. Locus names are taken from the `ID` column, or are written as `CHROM:POS` where `ID` is `.`.

A VCF names its samples but does not say which population each sample belongs to, so `--samples` is required with VCF input. It takes a file of two whitespace-separated columns, the sample name and then the grouping name:

```
# sample        grouping
HG00096         GBR
HG00097         GBR
NA18939         JPT
```

Anything after a `#` is a comment, and any further columns are ignored, so an existing sample sheet can usually be used as it is. Groupings are numbered in the order they first appear in this file, which is also the order in which they appear in the output. A sample in the VCF that is not in the map is skipped, and a sample in the map that is not in the VCF is ignored. Both cases are reported as warnings with a count.

Because the missing-data fraction is measured against a sample's gene copies, `--tolerance` means the same thing here as for STRUCTURE input. It is the fraction of a grouping's gene copies that are not called at that locus (section 3.6).

```
adze --data cohort.vcf.gz --samples populations.tsv \
     --tolerance 0.05 --out-prefix cohort
```

NON_DATA_ROWS, NON_DATA_COLS and GROUP_BY_COL describe the STRUCTURE format, and they are ignored with a warning for VCF input.

3.5 Groupings

With STRUCTURE input, rows are grouped by the value in the label column given by `--group-col`. With VCF input, the groupings come from `--samples`. In both cases, groupings appear in the output in the order in which they are first seen. Any label column can be used, so a file with columns for population, region and continent can be analyzed at any of these levels without being rewritten. Similarly, the same VCF can be analyzed at several levels by using a different map file.

`--pops` restricts the analysis to a comma-separated list of groupings, and `--exclude-pops` removes groupings from it. Note that both change J, and so they also change which alleles count as private. Removing a grouping can make an allele private to another grouping that was not private before.

3.6 Missing data and locus filtering

An allele code equal to `--missing` (by default `-9`) is treated as unobserved and reduces N_j at that locus. Because a statistic is only reported at a given g when it is defined at every locus, a single locus with a small N_j limits the whole analysis.

`--tolerance` x drops any locus at which some grouping has more than a fraction x of its gene copies missing. Loci dropped in this way are listed in the `_deletedloci` file (section 5.4). Filtering removes loci in exchange for a larger MAX_G, and section 8.1 works through this trade-off on the example data.

The default is $x = 0.1$. Version 1.0 used $x = 1$ by default, which keeps every locus. For genome-wide data this is rarely what is wanted. A single locus at which one grouping has no scored gene copies reduces MAX_G to 1 and makes that grouping undefined at every g, so that a run can finish without reporting any statistics. `--tolerance 1` gives the version 1.0 behaviour. Note that a parameter file that sets TOLERANCE is not affected by the new default, and all version 1.0 parameter files set it.

There are two ways in which a grouping can become unusable, and ADZE reports both. First, a locus at which no allele is observed in any grouping contributes nothing. Such loci are counted and reported, and they are treated as undefined at every g. Second, a locus at which one grouping has fewer than g scored gene copies makes that grouping undefined at g, and one such locus is enough to make the grouping's mean undefined. For each grouping, ADZE therefore reports the smallest number of gene copies it has at any locus, how many loci have that number, and how many loci it has no scored copies at. When this leaves nothing usable for a grouping, the warning also says how to fix it. `--dry-run` reports the same information without computing any statistics. With `--legacy`, such a grouping does not appear in the result file at all, and the warning explains why. If filtering removes every locus, ADZE stops with an error rather than reporting statistics over no data.

3.7 How a run proceeds

Version 2.0 reads the data twice and does not hold the genotypes in memory.

The first pass counts gene copies. For each grouping and each locus, it records how many gene copies were scored, which is one integer per locus per grouping and nothing about which alleles were seen. This is enough to decide everything that depends on the data set as a whole: which loci fail the missing-data filter, how large g can be for each grouping and overall, where the windows fall, and what `--dry-run` should report. Note that the filter cannot be applied during the first pass, because it is measured against a grouping's total number of gene copies, which is only known at the end of the file.

The second pass computes the statistics. The loci are read one at a time, and for each locus the probabilities Q_{ijg} are computed once and used by every requested statistic (allelic richness, private allelic richness, and private allelic richness for each combination of populations). Each statistic keeps a running mean and sum of squared deviations for each g, and for each window at each g, so no per-locus values are stored. Once a locus has been written to the output, it is no longer held in memory.

As a result, peak memory does not grow with the number of genotypes. For example, on a VCF of 320 000 loci and 10 groupings, the version 2.0 build from before this change, which held the data in memory, used 195 MB, and the current version uses 32 MB. The difference grows with the number of loci.

`--dry-run` uses only the first pass and does not read the data a second time, so it is much cheaper than a full run.

structure input is converted first

A STRUCTURE file has one row per gene copy and one column per locus, so no locus can be computed until every row has been read. Such a file is therefore first converted into allele counts arranged by locus. The conversion reads a block of locus columns at a time, with the block size chosen to fit in a fixed amount of memory, and the second pass then reads the counts. Because the converted file holds counts rather than genotypes, it is much smaller than the data file. For example, an 80 000-locus file of 122 MB converts to 7.2 MB.

The converted file is written in the same directory as the output and removed when the run ends. If `--count-file` is given, the file is kept instead, and the next run over the same data reuses it. A kept file records the data file's path, size and modification time, the grouping filters, the missing-data code and the column layout. If any of these has changed, the file is not used, and ADZE reports the reason and converts the data again. `--convert-chunk` sets the number of loci converted in each pass.

The conversion has a cost. A STRUCTURE run reads its input twice and also writes and reads the converted file, so it is slower than before. On 320 000 loci it took 5.7 s, compared with 4.1 s for the build that held the data in memory, while using 133 MB rather than 216 MB. The memory saving grows with the number of loci (260 MB rather than 886 MB at 1.28 million loci). A VCF run only pays for the extra pass.

4 Parameters and options

4.1 Parameter file syntax

A parameter file is a sequence of lines, each a keyword followed by its value:

```
DATA_FILE small_data.stru
GROUP_BY_COL 5
MAX_G 8
```

```
TOLERANCE 0.24
# everything after a hash is a comment
```

Anything after a `#` is ignored. The value of a parameter is the rest of the line with the surrounding whitespace removed, so a value can contain spaces and can contain other keywords. Boolean parameters take `0` or `1`.

4.2 Reporting a single g

By default, every g from 1 to `MAX_G` is reported. `--at-g` reports a single g instead:

`--at-g` ***N***
: reports only $g = N$.

`--at-g max`
: reports only $g =$ `MAX_G`, so the value of `MAX_G` does not need to be known in advance.

This applies to every output. The three statistics, their per-locus files and their window files all report the single g, with the same columns as a full run (including the column for g), so they can be read in the same way.

The requested g is limited by two values: `MAX_G`, and the smallest number of gene copies scored at any locus, which is the largest g at which the statistics are defined. If the request is larger than the smaller of these two values, ADZE uses that value instead and prints a warning giving the value requested, the value used and which limit applied. Note that this is different from what happens when `MAX_G` is too large. In that case ADZE still reports every g up to `MAX_G` and gives `NA` for the values that cannot be computed, which still gives a useful result for the smaller values of g. A report at a single g that contained only `NA` would not be useful. A value below 1 is refused, because g is a sample size.

Reporting a single g is also faster. The probabilities Q_{ijg} are computed by a recurrence in g, so they must still be computed for every g up to the one being reported. However, most of the cost of a full run is in accumulating and writing the statistics at every g, and this is only done for the requested g. For example, on 20 000 loci in 12 groupings with all 66 pairs of groupings and `MAX_G` of 33, the full run took 3.42 s, `--at-g 2` took 0.60 s and `--at-g 33` took 0.65 s.

4.3 Option reference

The following list is generated from the table of options in the source code, so it lists exactly the options that the program accepts. Each entry gives the long option, the version 1.0 short flag and the parameter-file keyword where these exist, and the default value that is used when the option is not given. The default is defined once, in the same table, and the same value is printed by `--help` and listed in the README. `test/docs.py` fails if the README and the table disagree about a value.

4.3.1 Input options

`--data` *FILE*, `-f`, `DATA_FILE` **(paramfile)**
: *string*
: data file in STRUCTURE format or VCF, optionally gzipped (required)

`--format` *FMT*, `FORMAT` **(paramfile)**
string
input format: auto, structure or vcf (auto uses the file name) *Default:* `auto`.

`--samples` *FILE*, `SAMPLE_FILE` **(paramfile)**
string
map of samples to groupings, required for VCF (columns: sample grouping)

`--loci-map` *FILE*, `LOCI_MAP` **(paramfile)**
string
locus positions for STRUCTURE input (columns: locus chromosome position)

`--count-file` *FILE*, `COUNT_FILE` **(paramfile)**
string
keep the converted counts of a STRUCTURE file and reuse them *Default:* `temporary, removed`.

`--convert-chunk` *N*, `CONVERT_CHUNK` **(paramfile)**
int
number of loci per pass when converting STRUCTURE input *Default:* `as many as fit 64 MB`.

`--max-g` *N*, `-g`, `MAX_G` **(paramfile)**
int
largest standardized sample size *Default:* `the largest the data supports`.

`--non-data-rows` *N*, `-nr`, `NON_DATA_ROWS` **(paramfile)**
int
number of header rows before the genotypes *Default:* `1`.

`--non-data-cols` *N*, `-nc`, `NON_DATA_COLS` **(paramfile)**
int
number of label columns before the genotypes *Default:* `detected`.

`--loci` *N*, `-l`, `LOCI` **(paramfile)**
int
number of loci *Default:* `detected`.

`--data-lines` *N*, `-d`, `DATA_LINES` **(paramfile)**
int
number of data rows *Default:* `detected`.

`--group-col` *N*, `-s`, `GROUP_BY_COL` **(paramfile)**
int
label column that gives the grouping *Default:* `the last label column`.

`--missing` *STR*, `-m`, `MISSING` **(paramfile)**
string
code for a missing allele *Default:* `-9`.

`--pops` *LIST*
string
analyze only these groupings (comma-separated) *Default:* `every grouping`.

`--exclude-pops` *LIST*
string
analyze every grouping except these *Default:* `none excluded`.

4.3.2 Analysis options

`--hap-snps` *N*, `HAP_SNPS` **(paramfile)**
int
collapse N consecutive SNPs into haplotype alleles (phased VCF only) *Default:* `off`.

`--hap-bp` *N*, `HAP_BP` **(paramfile)**
int
collapse the SNPs in each N basepairs into haplotype alleles (phased VCF only) *Default:* `off`.

`--min-hap-snps` *N*, `MIN_HAP_SNPS` **(paramfile)**
int
drop haplotype blocks with fewer than N SNPs *Default:* `1`.

`--window-bp` *N*, `WINDOW_BP` **(paramfile)**
int
sliding windows of N basepairs (needs locus positions) *Default:* `off`.

`--window-loci` *N*, `WINDOW_LOCI` **(paramfile)**
int
sliding windows of N loci, after filtering (needs locus positions) *Default:* `off`.

`--step-bp` *N*, `STEP_BP` **(paramfile)**
int
step between basepair windows (default: the window size) *Default:* `the window width`.

`--step-loci` *N*, `STEP_LOCI` **(paramfile)**
int
step between locus windows (default: the window size) *Default:* `the window width`.

`--min-window-loci` *N*, `MIN_WINDOW_LOCI` **(paramfile)**
int
do not report windows with fewer than N loci *Default:* `1`.

`--at-g` *N*/*max*, `AT_G` **(paramfile)**
string
report only this g, or max for MAX_G *Default:* `every g`.

`--stat` *LIST*
string
which statistics to compute: richness,private,tuples *Default:* `richness,private`.

`--tolerance` *X*, `-t`, `TOLERANCE` **(paramfile)**
float
drop loci where any grouping has more than this fraction missing (1 keeps all) *Default:* `0.1`.

`--combinations`, `-c`, `COMB` **(paramfile)**
boolean
also compute private allelic richness for combinations of groupings *Default:* `off`.

`--tuples-k` *LIST*, `-k`, `K_RANGE` **(paramfile)**
string
combination sizes for –combinations, e.g. 2 or 1-3 or 1,3,5-7

`--tuples` *FILE*
string
compute only the combinations listed in FILE, one per line *Default:* `every k-subset`.

`--threads` *N*
int
number of threads; output does not depend on it *Default:* `1`.

4.3.3 Output options

`--out-prefix` *PREFIX*
string
start every output file name with PREFIX *Default:* `adze`.

`--out-richness` *FILE*, `-r`, `R_OUT` **(paramfile)**
string
allelic richness output file *Default:* `from --out-prefix`.

`--out-private` *FILE*, `-p`, `P_OUT` **(paramfile)**
string
private allelic richness output file *Default:* `from --out-prefix`.

`--out-tuples` *FILE*, `-o`, `C_OUT` **(paramfile)**
string
combinations output file (one per size k, with _k added) *Default:* `from --out-prefix`.

`--full-richness`, `-fr`, `FULL_R` **(paramfile)**
boolean
also write per-locus allelic richness *Default:* `off`.

`--full-private`, `-fp`, `FULL_P` **(paramfile)**
boolean
also write per-locus private allelic richness *Default:* `off`.

`--full-tuples`, `-fc`, `FULL_C` **(paramfile)**
boolean
also write per-locus values for combinations *Default:* `off`.

`--legacy`, `LEGACY_FORMAT` **(paramfile)**
boolean
write the version 1.0 layout (no header, spaces, undefined rows left out) *Default:* `off`.

4.3.4 Other options

`--params` *FILE*
string
read parameters from FILE (also accepted as the first argument)

`--dry-run`
boolean
report the data dimensions, groupings and largest MAX_G, then stop *Default:* `off`.

`--progress`, `-pp`, `PRINT_PROGRESS` **(paramfile)**
boolean
show progress bars *Default:* `on when stderr is a terminal`.

`--quiet`
boolean
do not print progress or informational messages *Default:* `off`.

`--tolerance-only`, `-tnocalc`, `TNC` **(paramfile)**
boolean
apply the missing-data filter, report it, and stop *Default:* `off`.

`--no-check`, `-skipchk`, `SKIP_CHK` **(paramfile)**
boolean
accepted for compatibility; the data file is always checked *Default:* `off`.

5 Output

By default, ADZE writes `adze.richness`, `adze.private` and `adze.summary.txt` in the current directory. `--out-prefix` replaces `adze` in these names. If a file name is given with `--out-richness`, `--out-private` or `--out-tuples`, that name is used as it is, and the version 1.0 suffixes (`_summary`, `_fulldata` and `_deletedloci`) are added to it.

Results are written to files. Progress messages, warnings and errors are written to standard error, so redirecting standard output captures only the results. `--quiet` turns off these messages. A progress bar is only shown when standard error is a terminal.

5.1 Allelic richness and private allelic richness

There is one row for each grouping at each g. Each row gives the grouping name, the value of g, the number of loci used, and then the mean, variance and standard error across those loci (Equations 6–8). The columns are separated by tabs and named on the first line, and a statistic that is undefined at that g is written as `NA`. The following lines are from `example/small_r`:

```
POP_GROUPING    G   NUM_LOCI    MEAN        VAR         STD_ERR
AMERICA         1   3           1           0           0
AMERICA         2   3           1.34524     0.0973639   0.180152
```

`--legacy` writes the version 1.0 layout instead (section 5.6).

5.2 Private allelic richness of tuples

These files have the same layout, with the names of the groupings in the combination in place of the single grouping name. With `--tuples-k`, one file is written for each combination size k, and k is added to the file name. With `--tuples`, all of the named combinations are written to one file.

5.3 Locus-specific output

`--full-richness`, `--full-private` and `--full-tuples` also write a `_fulldata` file, which gives the statistic at every locus. There is one row for each locus and grouping and one column for each g, the columns are separated by tabs, and the first line names them. A value that is undefined at a given g is written as `NA`. The following example is from a small data set of four loci and two groupings, in which `POP1` has only two scored gene copies at `L2`:

```
LOCUS   POP_GROUPING    G1      G2      G3
L0      POP0            1       1.53571 1.80357
L0      POP1            1       1.25    1.375
   ...
L2      POP0            1       1.53571 1.80357
L2      POP1            1       1       NA
   ...
```

The locus is in the first column because the loci are read one at a time, and the values for every grouping at a locus are computed before the next locus is read (section 3.7). The rows for a locus are therefore written together.

This layout is the transpose of the version 1.0 layout, and it is the main change to the output format in version 2.0. Version 1.0 wrote one row for each grouping and g, with one column for each locus and the summary statistics at the end of the row. For the same data, version 1.0 wrote:

```
POP_GROUPING G NUM_LOCI L0 L1 L2 L3 MEAN VAR STD_ERR
POP0 2 4 1.53571 1.42857 1.53571 1.53571 1.50893 0.0028699 0.0267857
POP0 3 4 1.80357 1.64286 1.80357 1.80357 1.76339 0.00645727 0.0401786

POP1 2 4 1.25 1.53571 1 1.42857 1.30357 0.0548469 0.117097
```

With genome-wide data, the version 1.0 layout gives a few rows of a million or more fields each, which is more columns than spreadsheets and many other tools can read. In addition, such a file cannot be written until every locus has been computed, which required the per-locus values at every g to be held in memory (section 10). With one row per locus, the file is written as the loci are read. The summary statistics are not repeated in the `_fulldata` file, since they are in the statistics file. The values for one grouping at one g can be extracted as a column, for example for `POP1` at $g = 2$:

```
awk -F'\t' '$2 == "POP1" {print $1, $4}' out.richness_fulldata
```

In this example, the allelic richness of `POP1` at `L2` is undefined at $g = 3$, so its mean across loci at $g = 3$ is also undefined. Version 2.0 writes `NA` in that cell, and the statistics file reports `NA` for `POP1` at $g = 3$. Version 1.0 omitted the row for `POP1` at $g = 3$ from both files, without saying why. Note that `--legacy` only applies to the statistics files. The `_fulldata` files are written in the same way in both layouts.

5.4 Filtered loci

When `--tolerance` is below 1, a `_deletedloci` file gives the number of loci that were dropped, the threshold that was applied, and the name of each dropped locus, in the order in which the loci appear in the input. Version 1.0 listed them in descending order of position. The order changed because each dropped locus is now written as it is read, instead of the names being collected first. The file is written even when every locus is dropped, since that is the case in which the list is most needed.

5.5 Run summary

A `_summary` (or `.summary.txt`) file is written on every run. It lists the parameters that were used, including values that were measured from the data and the value of `MAX_G` when it was determined from the data, and it records the elapsed time at which each step of the run finished.

5.6 Version 1.0's layout

The statistics files are tab-separated with a header row, and a statistic that is undefined at a given g is written as `NA`.

`--legacy` writes the statistics files in the version 1.0 layout. In this layout there is no header, the columns are separated by spaces, groupings are separated by a blank line, and a row that is undefined at a given g is left out rather than marked. Leaving out these rows is the reason the default changed. In the version 1.0 layout, a missing row could mean either that the statistic could not be computed or that it was not requested. In addition, a grouping that is undefined at every g does not appear in the file at all, and one poorly scored locus is enough to cause this (section 3.6). A run can then finish and report nothing for that grouping without any explanation in the file. Only one of the two layouts can be written in a run.

In the default layout, the names of the groupings in a combination are joined with commas, so that a combination stays in a single tab-separated column. In the version 1.0 layout they are joined with spaces.

The window files (section 6) are always tab-separated with a header, whichever layout is used, since they are new in version 2.0. The `_fulldata`, `_deletedloci` and `_summary` files are also not affected by `--legacy`. Note in particular that the `_fulldata` files are transposed relative to version 1.0 in both layouts (section 5.3), so `--legacy` does not reproduce these files.

5.7 Exit status

0	success
2	usage error: unknown option, missing value, contradictory request
3	I/O error: a file could not be opened, read or written
4	data error: the file disagrees with itself or with the parameters

6 Sliding windows

Every statistic in section 1 can be reported in windows along the genome, in addition to being reported once for the whole data set.

6.1 What a windowed statistic is here

For each window, ADZE reports the same mean, variance and standard error over the loci in the window that it reports over all loci. The estimators do not change. $\hat{\alpha}_g^{(j)}$ and $\hat{\pi}_g^{(j)}$ are expectations at a single locus, and a window only changes the set of loci that they are averaged over.

This has two consequences. First, the windowed statistics do not use any information about linkage between the loci in a window. A windowed statistic is an average over nearby loci, and it does not measure haplotype structure in the window. For alleles that are haplotypes, see section 7. The two can be combined, in which case each window averages over the haplotype blocks it contains. Second, the variance and standard error are computed across the loci in the window, so they are

noisy when a window contains few loci. With one locus, the variance is undefined and ADZE writes `nan`. `--min-window-loci` sets the smallest number of loci a window must contain to be reported.

6.2 Coordinates

Because a window is a region of the genome, every locus must have a position. With VCF input, the positions are taken from `CHROM` and `POS`. The STRUCTURE format has no positions, so they are given in a separate file with `--loci-map`:

```
# locus   chromosome   position
L0        chr1         5000
L1        chr1         10000
L2        chr2         5000
```

The columns are separated by whitespace, a `#` starts a comment, and any further columns are ignored. Every locus in the data must be in this file. A locus without a position stops the run with an error, since a window over loci whose positions are not known would not correspond to a region of the genome. The loci must also be in genome order, with positions increasing within a chromosome and all loci on a chromosome together. If they are not, ADZE reports the first locus that is out of order rather than sorting the data.

6.3 Defining the windows

The size of a window is given either in basepairs or in loci:

`--window-bp` ***N***
: gives windows that are N basepairs wide, starting at position 1 and moving by `--step-bp`. The windows do not depend on where the loci are, so the same region gives the same windows in every run, and two data sets can be compared window by window. The number of loci in a window varies with the density of loci.

`--window-loci` ***N***
: gives windows of N loci that remain after filtering, moving by `--step-loci`. Each window has the same number of loci, and its width in basepairs varies with the density of loci. The reported interval runs from the first to the last locus in the window.

By default, the step is equal to the window size, so the windows do not overlap. A step smaller than the window size gives overlapping (sliding) windows. No window spans two chromosomes. Windows are laid out over the loci that remain after missing-data filtering (section 3.6), so a locus that is dropped there is not in any window, and a window of N loci contains N loci that remain after filtering.

`--min-window-loci` N leaves out windows that contain fewer than N loci. The default of 1 reports every window that contains at least one locus. Windows that contain no loci are never reported.

6.4 g is fixed across the scan

`MAX_G` is determined from the whole genome, as it is for the statistics over the whole data set. It is the largest standardized sample size that every grouping supports at every locus that remains after

filtering, and it is not determined separately for each window. A window in a region with larger sample sizes is therefore not reported at a larger g than the rest of the genome, so that all windows can be compared with each other. Because a set of loci is undefined at g if any locus in it cannot reach g, this also means that with the default `MAX_G` every window is defined at every reported g. If `MAX_G` is set larger than some locus supports, the windows containing that locus are written as `NA` rather than left out, so the file still has one row for each window at each g.

6.5 Output

Each requested statistic writes an additional file, with `_windows` added to its name. For example, with `--out-prefix scan`, `scan.richness` is accompanied by `scan.richness_windows`. The files for the whole data set are the same whether or not windows are requested.

Window files always have a header and are always tab-separated, even with `--legacy`, since they are new in version 2.0. The columns are

```
CHROM  START  END  POP_GROUPING  G  NUM_LOCI  MEAN  VAR  STD_ERR
```

In the file for combinations of populations, `TUPLE` takes the place of `POP_GROUPING`, and the grouping names are joined with commas so that each combination is in one column. The rows are ordered by grouping, then by g, and then by position along the genome, so the rows for one grouping at one g are together.

A genome scan is usually examined at a single g, and `--at-g` (section 4.2) reports only that g. The file then has one row for each window and grouping, rather than one for each window, grouping and g. For the scan in section 8.4, this is 48 rows rather than 1056.

Windowed output for combinations of populations can be large, since there is one row for each window, g and combination. `--dry-run` reports the number of rows each statistic would write, the number of windows, and how many windows contain too few loci to be reported, before any statistic is computed.

7 Haplotype alleles

The statistics in section 1 count alleles at a locus. With phased data, they can instead count haplotypes across a short run of loci. `--hap-snps` and `--hap-bp` collapse consecutive SNPs into blocks, and the alleles of a block are the haplotypes carried by its gene copies.

```
adze --data cohort.vcf.gz --samples pops.tsv --hap-snps 5 --out-prefix hap
adze --data cohort.vcf.gz --samples pops.tsv --hap-bp 10000 --out-prefix hap
```

The estimators do not change. $\hat{\alpha}_g^{(j)}$ and $\hat{\pi}_g^{(j)}$ are sums over the alleles at a locus, and a block is treated as a locus whose alleles are haplotypes. The rarefaction, the averaging across loci and the output are the same as described in the rest of this manual. However, because an allele is now a haplotype, there are some additional requirements and caveats, which are described below.

7.1 What a haplotype allele is here

For each gene copy, ADZE reads its allele at every SNP in the block, in order along the chromosome, and treats this sequence as one allele. Two gene copies carry the same haplotype allele if they

match at every SNP in the block. The blocks do not overlap, so each SNP belongs to exactly one block and no gene copy is counted twice.

`--hap-snps` N makes each block N consecutive SNPs, so every block has the same number of SNPs, and its width in basepairs varies with the density of SNPs. `--hap-bp` N makes each block a region of N basepairs, starting at position 1 and moving by N. The same region therefore gives the same blocks in every run, so two data sets can be compared block by block, and the number of SNPs in a block varies instead. A block never spans two chromosomes, a region that contains no SNP gives no block, and `--min-hap-snps` drops blocks that contain too few SNPs.

7.2 Phase is required

A haplotype is a run of alleles along one chromosome, so the input must say which chromosome copy each allele is on. ADZE therefore requires phased genotypes. Every `GT` with more than one allele must use `|`, and a record that uses `/` stops the run with an error that gives the record and the sample. Haploid gene copies are phased by definition. Note that ADZE does not phase the data or check the quality of the phasing. A haplotype assembled from an unphased genotype would be an arbitrary pairing of alleles, which is why an unphased genotype is an error rather than a warning.

For the same reason, the STRUCTURE format is not supported in this mode, since it does not say which chromosome copy an allele is on. Giving `--hap-snps` or `--hap-bp` with STRUCTURE input is refused as a usage error.

The records must also be sorted by chromosome and then by position. A block is a run of consecutive records, so if the records were out of order a block would join SNPs that are not next to each other. An ordinary run only needs sorted input for sliding windows, but a haplotype run always needs it. The run stops at the first record whose position is smaller than the record before it, or that returns to a chromosome that has already ended (`bcftools sort` fixes either problem).

7.3 One uncalled genotype makes the block uncalled

A gene copy only carries a haplotype allele in a block if it is called at every SNP in the block. If it is uncalled at any SNP in the block, it does not contribute to that block. It is not given a partial haplotype, since this would be an allele that no gene copy carries, and it would increase every count that included it. A gene copy whose `GT` at some record has fewer alleles than the sample's ploidy is also counted as uncalled there.

As a result, missing data accumulate along a block. If each SNP is called independently with probability $1 - m$, a gene copy is callable across a block of N SNPs with probability about $(1 - m)^N$. For example, with 2% missing data, a block of 50 SNPs keeps about a third of its gene copies. Because N_j enters the rarefaction directly, this reduces the largest g the data can support. `--tolerance` applies to blocks in the same way as to loci, so a block where some grouping has lost more than the allowed fraction of its gene copies is dropped. The `_blocks` file (section 7.5) gives the number of callable gene copies in each grouping for every block.

7.4 Richness saturates as blocks lengthen

When a block is long enough, every gene copy carries a different haplotype. If a block has as many distinct haplotypes as callable gene copies, no haplotype is repeated, every subsample of g gene copies contains g distinct haplotypes, and $\hat{\alpha}_g^{(j)} = g$ regardless of the data. In this case the statistic no longer measures anything about the populations. ADZE counts these blocks and reports the number at the end of the run, and the `HAPLOTYPES` column of the `_blocks` file shows how close each block is to this limit.

Private allelic richness has the same limit, because a haplotype carried by only one gene copy is private to that gene copy's grouping. For example, in simulated biallelic data in which both alleles at every SNP have a frequency between 0.2 and 0.8 in every grouping, so that almost no single-SNP allele is private, the mean private allelic richness at $g = 2$ was 0.0025 for blocks of 1 SNP, 0.11 for 2 SNPs, 1.42 for 5 SNPs, 1.98 for 10 SNPs and 2.0 for 20 or 50 SNPs. We therefore recommend using the shortest blocks that answer the question, and comparing results across several block lengths.

7.5 Naming and output

A block with more than one SNP is named `chromosome:start-end`. A block with exactly one SNP takes the name of that record, so `--hap-snps 1` gives exactly the same output as an ordinary run. All of the output files described in section 5 are written, with blocks in place of loci. This includes the `_fulldata` files, and the window files in a windowed run.

One additional file, `_blocks`, describes the blocks themselves:

```
BLOCK           CHROM START END   SNPS HAPLOTYPES KEPT POP0 POP1 POP2 POP3
chr1:1000-4000  chr1  1000  4000  4    14         no   9    5    8    8
chr1:5000-8000  chr1  5000  8000  4    11         yes  12   8    7    8
```

There is one row for each block, in the order of the input file. Each row gives the number of callable gene copies in each grouping and whether the block was kept by the filter. Two runs can give the same mean richness with blocks that have very different numbers of haplotypes, and this file shows the difference.

7.6 Haplotype blocks in sliding windows

Haplotype blocks can be used with sliding windows. Each window averages over the blocks that it contains, in the same way that an ordinary windowed run averages over loci (section 6), and `--window-loci` counts blocks. A block is placed in a window by its first SNP. This has two consequences.

With `--window-loci`, the reported interval of a window runs from the first SNP of its first block to the last SNP of its last block, so it covers all of the SNPs in the window.

With `--window-bp`, a block belongs to the window that contains its first SNP, and it can extend past the end of that window. Its haplotypes then include SNPs outside the reported interval of the window. ADZE counts these blocks and reports the number. Blocks always fall inside windows when `--hap-bp` h divides both `--window-bp` and `--step-bp`, since both are counted from position 1. For example, the following run has no blocks that extend past their window:

```
adze --data cohort.vcf.gz --samples pops.tsv --hap-bp 5000 \
     --window-bp 100000 --step-bp 25000 --out-prefix scan
```

Blocks from `--hap-snps` do not have a fixed width, so they cannot be made to fall inside windows in this way. With typical SNP densities, a block is short compared with a window, and only the blocks at the edge of a window are affected.

`--dry-run` reports the blocks that a given block size produces, the number of SNPs they contain and their widths. It uses only the first pass over the data, so it is a fast way to choose a block size before running a full genome scan.

8 Examples

8.1 The distributed example data set

In this example we use the small data set in Figure 1, which has five loci and three groupings, to work through the basic operation of ADZE. Running ADZE with only the data file and the grouping column,

```
adze --data small_data.stru --group-col 5 --out-prefix small
```

reports the following, among other messages, on standard error:

```
Read 22 gene copies at 5 loci in 3 groupings (d:h:m:s 0:0:0:0.00)
2 loci have at least one grouping with more than 10% missing data.
MAX_G not given; using 6, the largest the 3 surviving loci support.
```

Two of the five loci have more missing data than the default tolerance of 0.1 allows, so they are dropped, and the analysis runs to $g = 6$ over the three loci that remain. If instead every locus is kept,

```
adze --data small_data.stru --group-col 5 --out-prefix keep --tolerance 1

MAX_G not given; using 3, the largest the 5 surviving loci support.
```

the largest standardized sample size is halved. The reason is missing data rather than sample size. The `EUROPE` grouping has six gene copies in total, but three of them are missing at the fourth locus (`D9S1825`), which leaves $N_j = 3$. The fourth locus is also missing 25% of the data in the `AMERICA` grouping, and the second locus (`D14S1007`) is missing 25% of the data in the `EAST_ASIA` grouping. These are the two loci that the default tolerance drops.

Which of the two runs to use depends on the data. The second run uses all five loci, but it only compares populations at $g \leq 3$. The first run compares them over a wider range of g, but with only three loci. The default tolerance favors the larger g, because a single locus that is poorly scored in one grouping limits g for every grouping.

The parameter file distributed with the example, `small_paramfile.txt`, runs the same analysis in the version 1.0 style. It gives all four dimensions of the data file, along with `MAX_G 8` and `TOLERANCE 0.2`:

```
adze small_paramfile.txt
```

Its results agree with the first run above wherever the two overlap, and the private allelic richness files are identical. The allelic richness files differ. With `MAX_G 8`, `AMERICA` and `EAST_ASIA` are also reported at $g = 7$ and 8, which their own sample sizes allow, and `EUROPE` is reported as `NA` at those values of g. The default stops at the largest g that every grouping supports. Private allelic richness stops at $g = 6$ in both runs, since private allelic richness can only be computed at a g that every grouping can reach.

8.2 A larger data set

The next example uses a larger data set with 1056 individuals from 52 populations across 377 microsatellite loci. This data file is from the Rosenberg et al. (2002) study and can be downloaded at `https://rosenberglab.stanford.edu/diversity.html`. Because calculating private alleles for all combinations at the population level would be very time consuming, for the purposes of this

example we run ADZE at the level of geographic regions. The region is in the fifth label column of the file, and grouping by region reduces the number of groupings from 52 to 7.

The version 1.0 parameter file for this analysis (Figure 3 of the version 1.0 manual) gave `MAX_G 70`, `DATA_LINES 2112`, `LOCI 377`, `NON_DATA_ROWS 1`, `NON_DATA_COLS 5`, `GROUP_BY_COL 5`, `K_RANGE 2-6` and `TOLERANCE 1`. In version 2.0, the four dimensions are measured from the file, so the same analysis is run with

```
adze --data diversitydata.stru --group-col 5 --max-g 70 \
     --stat richness,private,tuples --tuples-k 2-6 \
     --out-prefix hgdp
```

Figure 2 gives the mean allelic richness against g, Figure 3 the mean private allelic richness against g, and Figure 4 the mean private allelic richness for combinations of six of the seven regions ("missing" alleles, Equation 5). These figures are reproduced from the version 1.0 manual and were made with version 1.0. Version 2.0 computes the same values.

Africa has the highest allelic richness and by far the highest private allelic richness. The Americas and Oceania have the highest missing allelic richness at larger values of g (above about $g = 12$). In other words, many alleles found in all other regions are absent from these two regions.

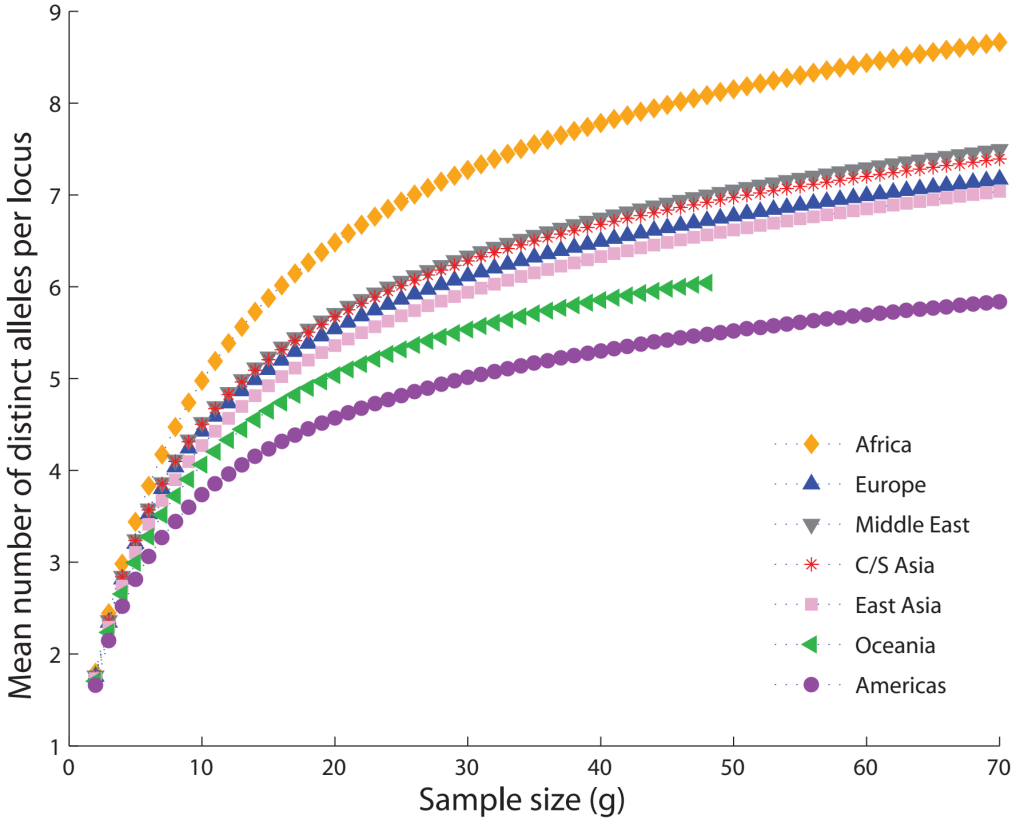


Figure 2: The mean number of distinct alleles per locus as a function of standardized sample size for seven major geographic regions.

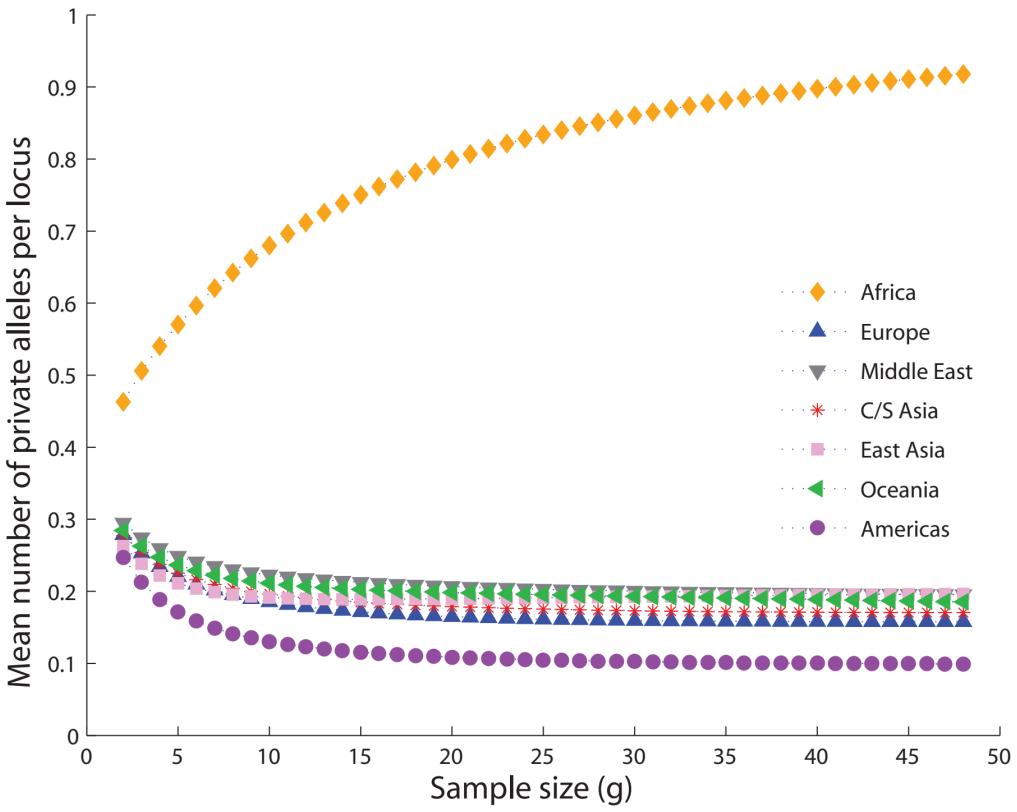


Figure 3: The mean number of private alleles per locus as a function of standardized sample size for seven major geographic regions.

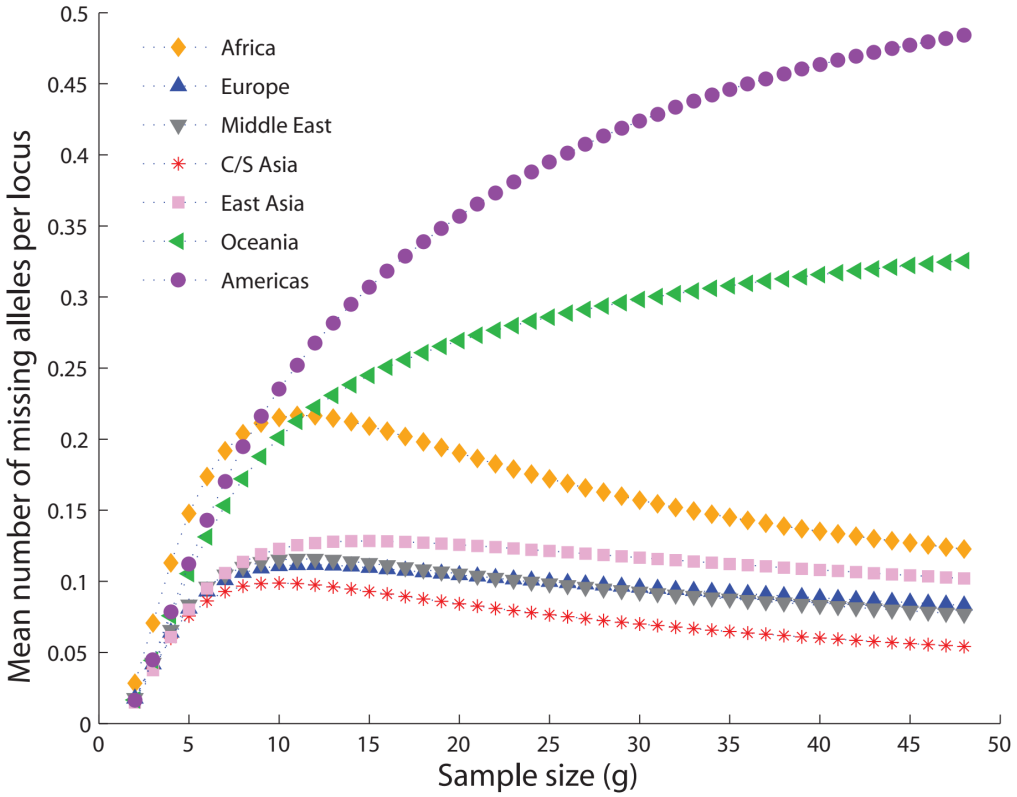


Figure 4: The mean number of missing alleles per locus for seven major geographic regions (private to combinations of six of the seven regions) as a function of standardized sample size.

8.3 Naming the tuples of interest

Computing private allelic richness for every combination of size k requires $\binom{J}{k}$ combinations for each k, which is $2^J - 1$ combinations if every k is requested. When only a few combinations are of interest, they can be listed in a file, one combination per line:

```
# tuples.txt
AMERICA, EUROPE
EAST_ASIA
AMERICA, EUROPE, EAST_ASIA

adze --data small_data.stru --group-col 5 --tuples tuples.txt \
     --stat tuples --out-prefix regions
```

Each line is a list of grouping names, separated by commas or spaces, as they appear in the data file, and a `#` starts a comment. Only the listed combinations are computed, and they can be of different sizes.

8.4 A genome scan

The script that generates the test data also writes positions for each locus, so this example can be reproduced from the repository alone. The following command makes 60 loci spaced 5 kb apart on two chromosomes, in four groupings of twelve diploid individuals:

```
python3 -c "import sys; sys.path.insert(0,'test'); import gen_data
gen_data.write_pair('scan', npops=4, nind=12, nloci=60, nall=6,
                    missing=0.03, seed=3, chroms=2, spacing=5000)"
```

Before running the scan, `--dry-run` reports what 50 kb windows with a step of 25 kb would contain:

```
adze --data scan.vcf --samples scan.samples \
     --window-bp 50000 --step-bp 25000 --dry-run
```

```
Largest feasible MAX_G: 20
MAX_G to be used:       the largest feasible
  (TOLERANCE 0.1 will drop loci first, which can raise both numbers)
Windows:          12 of 50000 bp, step 25000
  loci per window: 4-10
  richness rows:   960
  private rows:    960
```

There are twelve windows, and before filtering g can go up to 20, because every grouping has 24 gene copies and the locus with the most missing data has 20. Note that the dry run reports these numbers before filtering. Filtering drops loci first, which can increase both the largest g and the number of rows, as the full run then reports. The scan is then run, here requiring at least three loci in each window:

```
adze --data scan.vcf --samples scan.samples \
     --window-bp 50000 --step-bp 25000 --min-window-loci 3 \
     --out-prefix scan
```

```
7 loci have at least one grouping with more than 10% missing data.
Laid out 12 windows over 53 loci.
MAX_G not given; using 22, the largest the 53 surviving loci support.
```

The filter drops seven of the 60 loci, which increases the largest g from 20 to 22, and the window file has 1056 rows. Adding `--at-g 10` reports only $g = 10$, which gives 48 rows, one for each window and grouping:

CHROM	START	END	POP_GROUPING	G	NUM_LOCI	MEAN	VAR	STD_ERR
chr1	1	50000	POP0	10	8	3.45471	0.0842792	0.10264
chr1	25001	75000	POP0	10	8	3.56664	0.0605046	0.0869659

Consecutive windows overlap by 25 kb, so each locus that is not at the end of a chromosome is in two windows. `scan.richness` holds the statistics for the whole data set, which are the same as in a run without windows. The rows at $g = 10$ are identical to the rows for $g = 10$ in the full run.

9 Performance and threading

`--threads` N divides the work on the loci among N threads. The loci are read in order, a batch at a time, and each locus in a batch is parsed and its statistics computed on whichever thread is free. The values are then added to the averages, and the output for each locus is written, in the order of the loci in the file. A run therefore writes exactly the same files with any number of threads, and `test/threads.py` checks this at 1, 2, 3, 4, 8 and 64 threads. The first pass over a VCF and the conversion of a STRUCTURE file also use the threads. The default is one thread.

Table 1 gives the time taken with different numbers of threads. The gain is largest when most of the time is spent computing the statistics, as for combinations of populations or for the `_fulldata` files. It is smallest when most of the time is spent in the steps that are still done on one thread, which are reading the data, adding the values to the averages, and writing the window files after the last locus. Memory use grows by a few megabytes with the number of threads.

Run	1 thread	2 threads	4 threads	8 threads
SNPs	2.68 s	1.58 s	1.01 s	0.93 s
SNPs, 45 pairs of groupings	7.06 s	4.02 s	2.51 s	2.11 s
SNPs, `_fulldata` files	15.08 s	8.15 s	4.72 s	3.91 s
SNPs, 100 kb windows	7.19 s	6.13 s	5.52 s	5.50 s
Microsatellites	2.18 s	1.23 s	0.77 s	0.69 s
Microsatellites, 120 triples	17.18 s	9.04 s	4.92 s	3.46 s

Table 1: Elapsed time with `--threads`, measured on an 8-core machine, best of two runs. The SNP data are 200 000 biallelic SNPs in a VCF, with 10 groupings of 20 diploid individuals and `MAX_G` of 40. The microsatellite data are 20 000 loci with 20 alleles each in the STRUCTURE format, with 10 groupings of 25 diploid individuals and `MAX_G` of 40. The windows are 100 kb wide with a step of 25 kb.

Compressed input is decompressed as it is read, so a `.vcf.gz` file takes more processor time to read than the same data uncompressed. On a busy file system it usually takes less elapsed time, since much less data has to be read from disk.

Table 2 gives the effect of the version 2.0 changes on the steps that take the most time in large runs, measured against a version 1.0 build on the same inputs. In version 1.0, the time for locus filtering grew with the square of the number of loci, and the time for the sweep over g grew with the square of `MAX_G`. In version 2.0, both grow linearly.

Phase	Condition	Version 1.0	Version 2.0
Locus filtering	8000 loci, 4623 dropped	6.00 s	<0.01 s
Richness and private	1000 loci, `MAX_G` 80	4.80 s	0.09 s
Combinations	12 groupings, 495 combinations	4.34 s	0.37 s
Reading	64 MB input	3.68 s	0.78 s
Peak memory	64 MB input	413 MiB	27 MiB

Table 2: Measured with `test/bench.py` on an 8-core machine, best of two runs. Version 1.0 held every genotype in memory, so its memory use grew with the sample size. Version 2.0 does not hold the genotypes in memory, so its memory use does not grow with the number of gene copies.

9.1 Haplotype blocks

Collapsing SNPs into blocks affects the cost of a run in two ways, and the balance depends on the block size. The counting pass keeps one count per block per grouping, so larger blocks mean fewer counts and less memory. However, the work at each block grows with the number of distinct haplotypes in the block, which increases with block size until every callable gene copy carries a different haplotype. The following table was measured on 40 000 phased biallelic SNPs, with 300 gene copies in 6 groupings, 2% of genotypes missing and `--max-g` 10:

`--hap-snps`	time	peak	blocks	haplotypes per block	callable copies	saturated blocks
off	0.29 s	7.4 MB	40 000	2.0	100%	—
1	0.45 s	8.1 MB	40 000	2.0	98%	0%
2	0.37 s	6.7 MB	20 000	4.0	96%	0%
5	0.45 s	5.6 MB	8 000	31.9	90%	0%
10	0.92 s	5.7 MB	4 000	210.7	82%	0%
20	0.45 s	5.5 MB	2 000	200.2	67%	96%
50	0.26 s	5.4 MB	800	109.4	37%	100%

The fraction of callable gene copies follows $(1-m)^N$ closely (98, 96, 90, 82, 67 and 37% measured, compared with 98, 96, 90, 82, 67 and 36% predicted), as expected from the rule in section 7 that one uncalled genotype makes the whole block uncalled. With blocks of 50 SNPs and 2% missing data, only a third of the gene copies are still callable, so two thirds of them are lost. The run time is largest for blocks of 10 SNPs, where the number of distinct haplotypes is high and there are still many blocks, and it decreases for longer blocks because there are fewer blocks. In addition, with blocks of 20 SNPs, almost every block has as many haplotypes as callable gene copies, so that $\hat{\alpha}_g^{(j)}$ equals g regardless of the data. Note that the fastest runs in the table are therefore the least informative.

10 Changes from version 1.0

10.1 Compatibility

Parameter files written for version 1.0 work without changes, as do the version 1.0 short flags. Two defaults differ. First, the default `--tolerance` is 0.1 rather than 1, so a run that does not set a tolerance now drops loci that version 1.0 kept. Every version 1.0 parameter file sets `TOLERANCE`, so these files are not affected, and `--tolerance 1` gives the version 1.0 behavior (section 3.6). Second, the statistics files are tab-separated with a header rather than in the version 1.0 layout, and `--legacy` writes the version 1.0 layout (section 5.6). A script that reads version 1.0 output therefore needs this flag, and the test that compares version 2.0 with version 1.0 uses it for this reason. Output files that are named explicitly keep the version 1.0 suffixes. `SKIP_CHK` and `--no-check` are still accepted but have no effect, since the data file is now always checked during the counting pass at no extra cost.

10.2 Interface

- Options can be given without a parameter file, and `--help` and `--version` work without any other arguments. Running ADZE with no arguments prints a usage message. Version 1.0 instead wrote a template parameter file into the working directory.
- The parameter file is read as a keyword followed by the rest of the line. Version 1.0 searched each line for any keyword, so a value that contained a keyword (for example, `R_OUT out_LOCI_richness`) was rejected, and values could not contain spaces.
- The dimensions of the data file are measured from the file (section 3.3).
- Messages and progress are written to standard error, and the exit status distinguishes usage errors, input and output errors, and data errors.
- VCF input, uncompressed or compressed with `gzip` or `bgzip`, is read with `--format` and `--samples` (section 3.4). Version 1.0 only read uncompressed STRUCTURE files.
- With phased VCF input, consecutive SNPs can be collapsed into haplotype blocks, and the statistics are then computed over haplotypes rather than single-SNP alleles (section 7).
- The genotypes are not held in memory. The data are read in two passes. The first counts gene copies and determines the filter, the largest g for each grouping, `MAX_G` and the windows. The second computes every statistic from one table of Q_{ijg} per locus (section 3.7). On a VCF of 1.28 million loci and 10 groupings, peak memory decreased from 764 MB to 145 MB with the same run time. For a STRUCTURE file of the same size, peak memory decreased from 886 MB to 260 MB, and the run took about twice as long, because the file is first converted to counts by locus.
- `--dry-run` uses only the counting pass and does not read the data a second time. On 80 000 loci, its memory use decreased from 54 MB to 11 MB and its run time from 0.47 s to 0.19 s.
- The `_deletedloci` list is in the order of the input rather than in descending order of position, and it is written even when every locus is dropped (section 5.4).
- Each statistic is still announced, but the statistics are computed together, so the run reports a single completion, and the summary has one `Statistics completed at` line rather than one for each statistic.

- The `_fulldata` files are transposed, with one row for each locus and grouping and one column for each g. Version 1.0 wrote one row for each grouping and g, with one column for each locus (section 5.3). Version 2.0 does not reproduce the version 1.0 layout of these files with either setting of `--legacy`.

- The statistics files are tab-separated with a header, and undefined rows are written as `NA` (section 5.6). In the `_fulldata` files, an undefined value is written as `NA` rather than the row being left out (section 5.3). `--legacy` writes the version 1.0 layout of the statistics files, which is byte-identical to the version 1.0 output except for the $g = 1$ rows.

- The default `--tolerance` is 0.1 rather than 1 (section 3.6). A grouping that is made unusable by a locus that is kept is reported by name, with the number of loci involved, rather than appearing only as missing results.

- Every statistic is now also reported at $g = 1$ (section 1.5), while version 1.0 started at $g = 2$. The default output therefore has one extra row for each grouping, and this is the only difference in the values reported.

- `--at-g` reports a single g instead of every g (section 4.2). A request that is too large is reduced to the largest g that can be computed, with a warning.

- Statistics can be reported in sliding windows (section 6) with `--window-bp` or `--window-loci` and the matching step, `--min-window-loci`, and `--loci-map`, which gives positions for STRUCTURE input. Version 1.0 reported each statistic once for the whole data set.

- New options: `--out-prefix`, `--stat`, `--pops`, `--exclude-pops`, `--tuples`, `--legacy`, `--dry-run`, `--threads` and `--quiet`.

10.3 Behaviour

The estimates do not change. The following behaviors differ, in each case because version 1.0 gave a result that could not be used:

1. A locus with no observed allele in any grouping is reported as undefined, with a warning. Version 1.0 stopped with a segmentation fault.

2. If filtering removes every locus, ADZE stops with an error. Version 1.0 continued and printed `nan` rows over zero loci.

3. A data row with the wrong number of columns is always an error. With checking turned off, version 1.0 read such a row incorrectly without a warning.

4. The variance at a single locus is reported as `nan` rather than as the result of dividing by zero.

5. Reported times are elapsed (wall-clock) times with two decimal places. Version 1.0 reported processor time rounded down to whole seconds, which is why the example run in the version 1.0 manual reports `0:0:0:0` throughout.

10.4 Verification

Every change was checked with a test that runs version 2.0 and a version 1.0 build on the same inputs and compares their output. The inputs are nine simulated data sets, which vary in the number of alleles, the amount of missing data, and the number of loci and groupings, and which include unequal sample sizes, haploid data and a locus with no observed alleles. Each data set is run with several parameter settings, and the distributed example is also included. All 69 cases that can be compared agree. In the remaining 14 cases, version 1.0 crashes or reports statistics over zero loci, so there is no output to compare. The test is in `test/regress.py`, and it needs a version 1.0 binary to compare against.

There are two qualifications on what it means for the outputs to agree.

- Version 1.0 does not report $g = 1$, so the files as a whole do not match. Every row at $g \geq 2$ is compared, and the $g = 1$ rows are checked separately. Each results file must have exactly one $g = 1$ row for each grouping or combination, so a build that stopped writing them would fail.

- The per-locus values, the labels and the numbers of loci are compared exactly. For the `_fulldata` files, the comparison is cell by cell, since the two layouts are transposes of each other (section 5.3). The mean, variance and standard error are compared with a relative tolerance of 10^{-9}, and values below 10^{-12} are treated as zero. Version 2.0 computes the mean and variance in a single pass as it reads the loci, rather than summing twice over stored per-locus values, so the last digits of a variance can differ. For example, on the distributed example version 1.0 reports a variance of 2.96×10^{-31} where every deviation is zero, and version 2.0 reports 0. The tolerance is four orders of magnitude smaller than the six significant digits that are printed, so a difference in any printed digit still fails the test.

Version 1.0 does not compute windowed statistics, so `test/windows.py` checks them in other ways. One window covering a data set on a single chromosome must give exactly the same output as the whole data set. The mean and variance of every window must match a recomputation from the per-locus values. The windows must be laid out as requested and must not span two chromosomes. Windows measured in basepairs and in loci must agree when the loci are evenly spaced. Finally, inputs over which a window cannot be defined must be refused.

Version 1.0 does not read VCF, so VCF input is checked by `test/formats.py`, which writes one set of genotypes as a STRUCTURE file, as a VCF and as a compressed VCF, and requires the three runs to give byte-identical output. Because both readers order the alleles in the same way, this comparison is exact. Any difference is therefore a real difference, and not a change in the order of a floating-point sum.

Haplotype blocks are checked by `test/haplotypes.py` against haplotypes assembled directly from the simulated data, without using ADZE. In addition, each block is rewritten as a single VCF record whose alleles are its haplotypes, and an ordinary run on this file must give the same output as the haplotype run on the original file. `--hap-snps 1` must give exactly the same output as an ordinary run.

References

Hurlbert, S. H. 1971. The nonconcept of species diversity: a critique and alternative parameters. *Ecology* 52: 577–586.

Kalinowski, S. T. 2004. Counting alleles with rarefaction: private alleles and hierarchical sampling designs. *Conservation Genetics* 5: 539–543.

Kalinowski, S. T. 2005. HP-RARE 1.0: a computer program for performing rarefaction on measures of allelic richness. *Molecular Ecology Notes* 5: 187–189.

Petit, R. J., A. El Mousadik, and O. Pons. 1998. Identifying populations for conservation on the basis of genetic markers. *Conservation Biology* 12: 844–855.

Rosenberg, N. A., J. K. Pritchard, J. L. Weber, H. M. Cann, K. K. Kidd, L. A. Zhivotovsky, and M. W. Feldman. 2002. Genetic structure of human populations. *Science* 298: 2381–2385.

Szpiech, Z. A., M. Jakobsson, and N. A. Rosenberg. 2008. ADZE: a rarefaction approach for counting alleles private to combinations of populations. *Bioinformatics* 24: 2498–2504.